

Security Assessment Report

Automated Vulnerability Discovery

Target Infrastructure: <http://testfire.net/>
Date of Assessment: 2026-05-19 13:08 IST
Reference ID: 7c5d7345-e243-4565-9014-14fddc9b94ed

1. Executive Summary

This document presents the findings of an automated security assessment conducted against **http://testfire.net/** on 2026-05-19 13:08 IST. The assessment utilized the SmartFuzz testing engine to evaluate the target against a predefined set of vulnerability classes. The overall risk posture of the application is currently evaluated as **Critical**.

Assessment Metric	Value
Total Payloads Generated	146
Total Findings Confirmed	3
Critical Risk Findings	1
High Risk Findings	0
Medium Risk Findings	1

2. Findings Summary

ID	Vulnerability	Parameter	Severity
1	SSRF	content	Critical
2	XSS	content	Medium
3	Info Disclosure	content	Low

3. Detailed Findings Analysis

Finding 1: SSRF in parameter 'content'

Severity:	Critical
Target URL:	http://testfire.net/index.jsp?content=http%3A%2F%2F169.254.169.254%2Flatest%2Fmeta-data%2F
Method:	GET
Status Code:	200
Response Time:	0.25 seconds
Injected Payload:	http://169.254.169.254/latest/meta-data/
Evidence:	<pre><!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet</pre>

Finding 2: XSS in parameter 'content'

Severity:	Medium
Target URL:	http://testfire.net/index.jsp?content=%26%23x3c%3Bscript%26%23x3e%3Balrt%28%27Altoro+Mutual+XSS%27%29%26%23x3c%3B%2Fscript%26%23x3e%3B
Method:	GET
Status Code:	200
Response Time:	0.265 seconds
Injected Payload:	<script>alert('Altoro Mutual XSS')</script>

Evidence:

```
<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
<head> <title>Altoro Mutual</title> <meta
http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" /> <link href="/style.css"
rel="stylesheet
```

Finding 3: Info Disclosure in parameter 'content'

Severity: Low

Target URL: <http://testfire.net/index.jsp?content=http%253A%252F%252F127.0.0.1%252F>

Method: GET

Status Code: 500

Response Time: 0.281 seconds

Injected Payload: <http%3A%2F%2F127.0.0.1%2F>

Evidence:

```
<!doctype html><html lang="en"><head><title>HTTP Status
500 - Internal Server Error</title><style
type="text/css">H1 {font-family:Tahoma,Arial,sans-serif;co
lor:white;background-color:#525D76;font-size:22px;} H2 {fo
nt-family:Tahoma,Arial,sans-serif;color:white;background-c
olor:#525D76;font-size:16px;} H3 {font-family:Tahoma,Arial
,sans-serif;color:white;background-color:#525D76;font-size
:14px;} BODY
```

4. Remediation Guidance

SSRF

1. Implement a strict allowlist of permitted destination domains, IP addresses, and protocols for any outbound server requests.
2. Actively block and reject network requests targeting internal, loopback, or reserved IP ranges (e.g., 10.0.0.0/8, 169.254.169.254, 127.0.0.1).
3. Disable HTTP redirection for server-side requests, or strictly re-validate the target URL against the allowlist after following any redirect.
4. Route all outbound application traffic through a dedicated egress proxy that explicitly drops internal network access.
5. Reference: OWASP Top 10 A10:2021 — Server-Side Request Forgery

XSS

1. Contextually output-encode all user-supplied data before rendering it in the browser (HTML, JavaScript, CSS, or URL contexts).
2. Implement a strict, restrictive Content-Security-Policy (CSP) HTTP response header to prevent the execution of inline scripts and untrusted sources.
3. Set the 'HttpOnly' and 'Secure' flags on all sensitive cookies (e.g., session tokens) to prevent theft via client-side scripts.
4. Sanitize any rich-text (HTML) input on the server side using a robust, actively maintained library (e.g., DOMPurify or Bleach).
5. Utilize modern frontend frameworks (React, Angular, Vue) that safely bind data to the DOM by default.
6. Reference: OWASP Top 10 A03:2021 — Injection

Info Disclosure

1. Sanitize and validate all user inputs.

Disclaimer

This report was generated automatically. Results should be reviewed and validated by a qualified security professional. False positives may be present.